

Unit 1: Computational Thinking & Algorithmic Logic • Topic 2

Problem Analysis, Algorithm Design

Learning to analyse a problem using the IPO model and design correct, well-structured algorithms.

Before a single line of code is written, a good programmer first understands the problem completely and then designs a step-by-step solution. This topic covers **problem analysis** (understanding exactly what needs to be solved) and **algorithm design** (planning precisely how to solve it).

2.1 What is Problem Analysis?

Problem analysis is the process of carefully studying a problem statement to identify what is being asked, what information is available, and what the expected result should be, *before* attempting a solution. Jumping straight to code without analysis usually leads to incomplete or incorrect programs.

The IPO Model

Almost every problem can be analysed using the simple **Input** → **Process** → **Output (IPO)** model:

Stage	Question to ask	Example (find the area of a rectangle)
Input	What data do I already have / need from the user?	length, breadth
Process	What steps/calculations convert input to output?	area = length × breadth
Output	What result must be produced / displayed?	the value of area

Steps in Problem Analysis

- 1 **Understand the problem statement** – read it multiple times; identify keywords.
- 2 **Identify inputs** – what data is given or must be taken from the user, and its data type.
- 3 **Identify the expected output** – what exactly should the program produce, and in what form.
- 4 **Identify constraints/assumptions** – e.g. ‘the number entered will always be positive’.
- 5 **Identify the processing steps** – the calculations, decisions, or repetitions needed to turn the input into the output.

WORKED EXAMPLE: Problem: ‘Find whether a student has passed, given marks out of 100.’
 Input: marks (integer). Process: compare marks with the passing threshold (say 40). Output: ‘Pass’ or ‘Fail’ message. Constraint: $0 \leq \text{marks} \leq 100$.

2.2 What is an Algorithm?

An **algorithm** is a finite, well-defined, ordered sequence of steps that takes some input and produces the desired output, in order to solve a specific problem. An algorithm is written in plain, structured language — it is independent of any programming language, so the same algorithm could be implemented in Python, C, or Java.

Characteristics of a Good Algorithm

- **Input** – zero or more well-defined inputs.
- **Output** – at least one well-defined output, related to the desired result.
- **Definiteness (Unambiguous)** – every step must be clear and have exactly one meaning.
- **Finiteness** – the algorithm must terminate after a finite number of steps; it cannot run forever.
- **Effectiveness** – every step must be simple and basic enough to be carried out, in principle, by a person using pencil and paper.

2.3 The Algorithm Design Process

Once the problem is analysed using the IPO model, algorithm design follows this general approach:

- 1 State the problem clearly.
- 2 List the inputs and outputs (from problem analysis).
- 3 Break the solution into smaller logical steps (decomposition).
- 4 Arrange the steps in the correct order using sequence, selection and iteration as needed.
- 5 Write the steps as numbered statements (the algorithm).
- 6 Verify the algorithm using a dry run / trace with sample data (Topic 3) before coding it.

2.4 Worked Examples of Algorithm Design

Example 1: Algorithm to find the largest of three numbers

```
Step 1: Start
Step 2: Read three numbers a, b, c
Step 3: If a > b and a > c, then largest = a
Step 4: Else if b > c, then largest = b
Step 5: Else, largest = c
Step 6: Print largest
Step 7: Stop
```

Example 2: Algorithm to find the sum of the first N natural numbers

```
Step 1: Start
Step 2: Read N
Step 3: Set sum = 0 and i = 1
Step 4: Repeat while i <= N:
    4.1: sum = sum + i
    4.2: i = i + 1
Step 5: Print sum
Step 6: Stop
```

Example 3: Algorithm to check whether a number is prime

```
Step 1: Start
Step 2: Read N
Step 3: Set i = 2 and is_prime = True
Step 4: Repeat while i <= N - 1:
    4.1: If N is divisible by i, set is_prime = False and exit the loop
    4.2: i = i + 1
Step 5: If is_prime is True and N > 1, print "Prime"
    Else print "Not Prime"
Step 6: Stop
```

2.5 A Note on Algorithm Efficiency

There can be more than one correct algorithm for the same problem. A good algorithm is not just correct — it should also be efficient, using as few steps, as little memory, and as little time as reasonably possible. At this stage, it is enough to be aware that different algorithms for the same problem can take different numbers of steps; a formal study of efficiency (time and space complexity) is done in later courses.

Practice Questions

A. Multiple Choice Questions

- The IPO model stands for:
(A) Input-Process-Output (B) Instruction-Program-Operation (C) Input-Program-Order (D) Idea-Plan-Output
- Which characteristic of an algorithm ensures it does not run forever?
(A) Definiteness (B) Effectiveness (C) Finiteness (D) Input
- An algorithm must have:
(A) Exactly one input and one output (B) Zero or more inputs and at least one output (C) No output at all (D) Ambiguous steps for flexibility
- Which of these is the FIRST step of problem analysis?
(A) Write the Python code (B) Understand the problem statement (C) Draw the flowchart (D) Test the program
- 'Every step of an algorithm must be simple enough to be performed by hand' describes:
(A) Finiteness (B) Effectiveness (C) Definiteness (D) Output

B. True / False

- An algorithm must be written in a specific programming language.
- Definiteness means every instruction in an algorithm has exactly one clear meaning.
- Problem analysis should be done after the program is fully coded.
- There can be more than one correct algorithm for solving the same problem.

C. Short Answer Questions

- Define an algorithm and list its five characteristics.
- Explain the IPO model with an example of your own (other than the ones in these notes).
- Why is problem analysis important before designing an algorithm?
- What is the difference between finiteness and effectiveness as properties of an algorithm?

D. Long Answer / Descriptive Questions

- Using the IPO model, analyse the problem 'calculate the simple interest given principal, rate and time', and then write a complete, numbered algorithm for it.
- Write an algorithm to find the smallest of three numbers. Explain, step by step, how your algorithm satisfies all five characteristics of a good algorithm.

E. Programming Problems

- Write an algorithm (numbered steps, not Python code) to check whether a given year is a leap year (a year is a leap year if it is divisible by 4, except that century years must be divisible by 400).
- Write an algorithm to reverse a given 3-digit number without using any built-in string reversal method (Hint: use the modulus and integer-division operators).
- Write an algorithm to calculate the factorial of a given number N using repetition (iteration).

Answer Key (A & B): MCQ: 1-A, 2-C, 3-B, 4-B, 5-B | T/F: 6-False, 7-True, 8-False, 9-True